

Autonomous 3D Printer Supply Chain

Three independent agents managing a volatile market

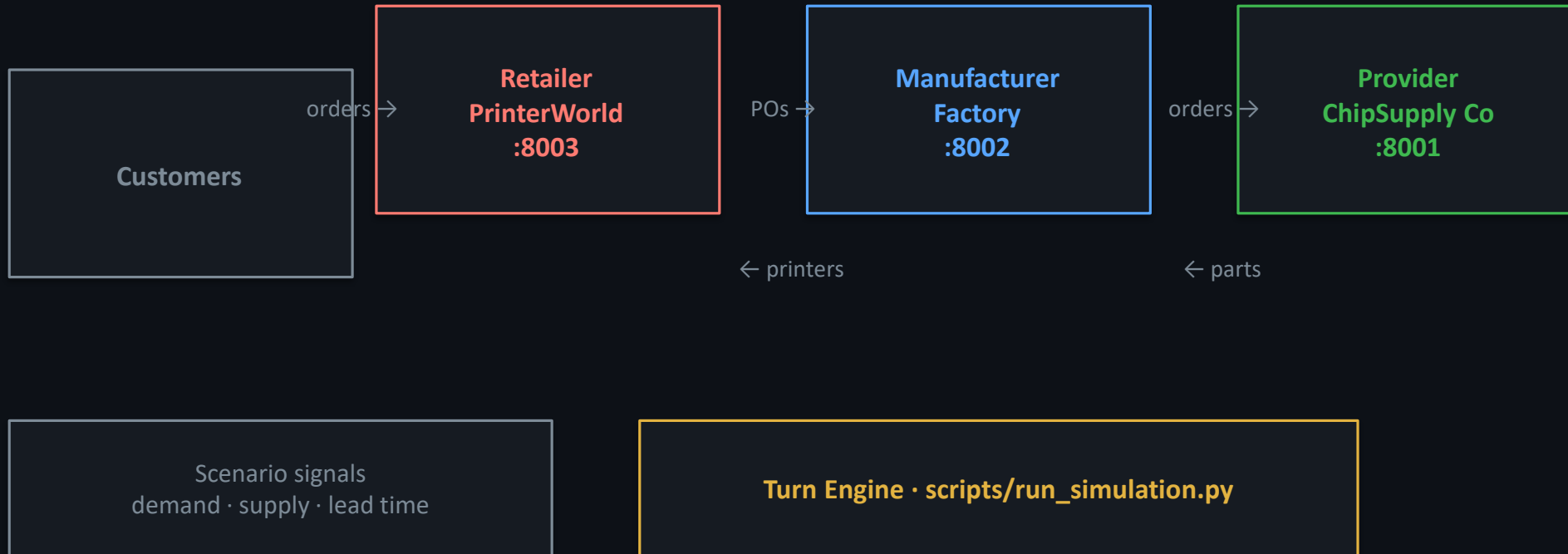
Provider

Manufacturer

Retailer

Demo scenario: holiday-rush

System Overview



Each app owns its own SQLite DB and exposes REST endpoints. The engine advances time, injects demand, and orchestrates all services in lock-step.

Turn Design — One Simulated Day

1. Read scenario signal for the day
2. Inject customer demand into the retailer
3. Provider reacts to stock pressure and supply conditions
4. Manufacturer releases orders, buys parts, adjusts wholesale prices
5. Retailer processes customers and buys printers
6. All three apps advance one day
7. Narrative log and metrics are saved

Why this order?

The upstream provider acts before the manufacturer places new pressure, then the manufacturer reacts to retail demand, and the retailer closes the loop with customers.

This order avoids stale-read race conditions between agents.

Agent Design: Provider

ChipSupply Co · :8001

Keep parts available while adjusting prices under supply pressure.

Inputs

- Current stock vs starting stock
- Open manufacturer orders
- Product price tiers
- Market signal: supply_modifier, lead_time_modifier, demand_modifier

Actions

- Restock products below threshold
- Raise prices during supply shortage
- Avoid stockouts on critical parts when possible

Design choice

- Prompt now provides RESTOCK flags and ready-to-run action hints
- Agent skips catalog/stock analysis
- Faster decisions, fewer wasted reasoning turns

Agent Design: Manufacturer

Factory · :8002

Convert retail purchase orders into finished printers.

Inputs

- Retail sales orders
- Raw-parts stock
- Finished-printer stock
- Supplier catalog
- Open part purchase orders
- BOM per printer model

Actions

- Release pending sales orders to production
- Order missing parts from provider
- Raise wholesale prices when utilisation/backlog is high

Important fix

- Buys against backlog + buffer, not only tiny shortages
- Avoids counting in_progress orders as new demand (BOM consumed)

Agent Design: Retailer

PrinterWorld · :8003

Serve customer demand and decide how many printers to buy.

Inputs

- Customer orders
- Retail stock
- Open purchases from manufacturer
- Wholesale prices
- Demand and price-sensitivity signals

Actions

- Process every pending customer order
- Backorder when stock unavailable
- Purchase enough printers to cover backlog + demand buffer
- Raise retail prices under stock pressure

Implementation note

- Made deterministic/direct for the long run
- LLM version hit max turns too often
- Keeps simulation stable and auditable

Scenario: Holiday Rush

Days 1–7 Pre-holiday normal — Demand $\times 1.0$ | Supply $\times 1.0$ | Lead time $\times 1.0$ — build buffer stock

Days 8–12 Black Friday — Demand $\times 3.0$ | Supply $\times 1.0$ | High price sensitivity

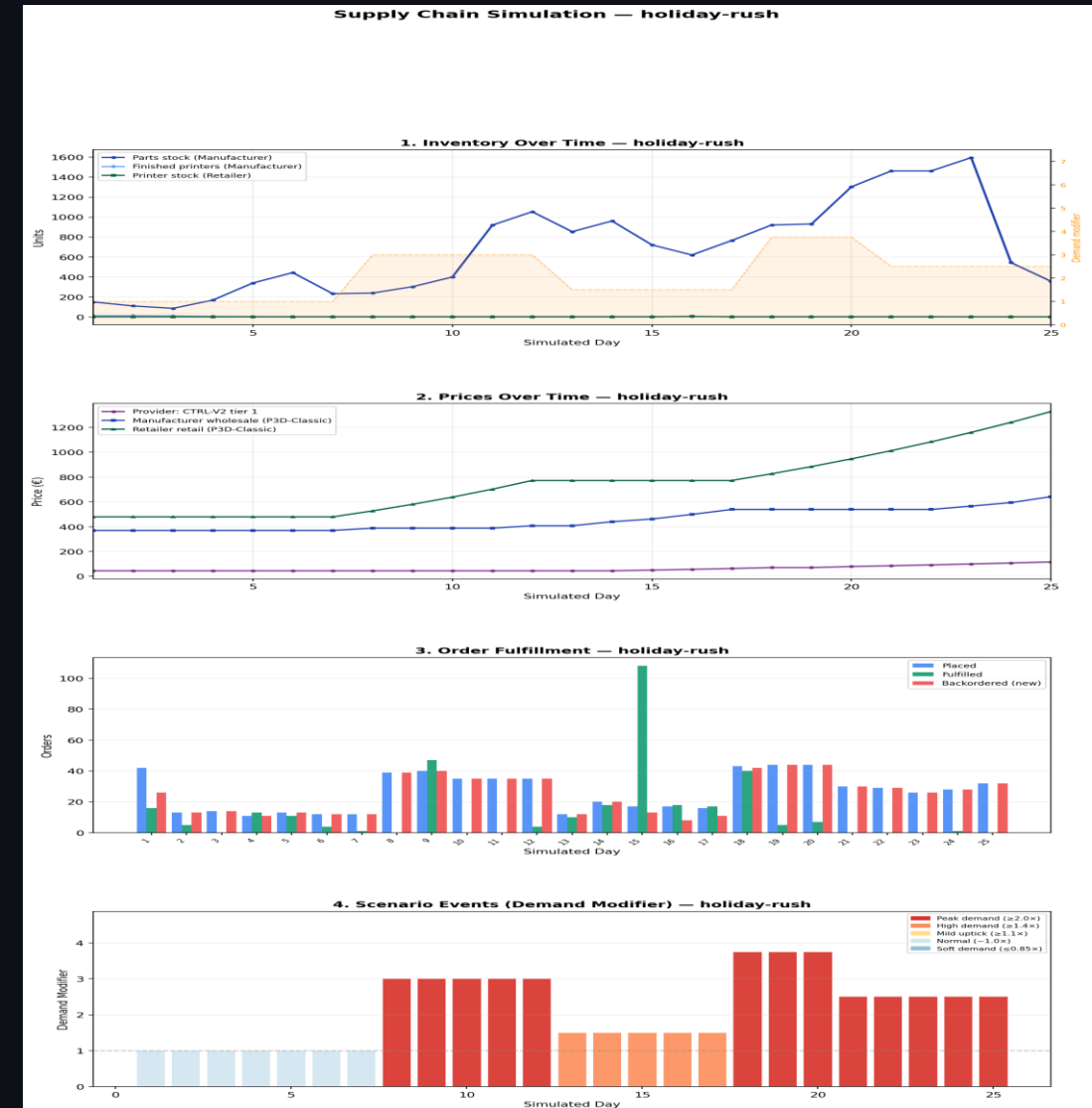
Days 13–20 Chip shortage — Demand $\times 1.5$ | Supply $\times 0.4$ | Lead time $\times 2.0$ — constrained upstream

Days 18–25 Christmas rush — Demand $\times 2.5$ | Supply $\times 0.6$ | Lead time $\times 1.5$ — overlaps shortage

⚠ **Days 18–20: Chip shortage + Christmas rush overlap — strongest stress test. Modifiers multiply.**

Holiday Rush — Run Observations

- Retailer stock reaches zero early and stays under pressure
- Backorders grow rapidly during Black Friday
- Manufacturer backlog high but causally reasonable
- Manufacturer places much larger part orders once demand spikes
- Provider repeatedly restocks kit_piezas back to its initial level (~120 units)
- Prices increase across the chain:
 - Provider raises kit_piezas unit price
 - Provider issues price raise-all during chip shortage
 - Manufacturer raises wholesale price under high utilisation



Emergent Behaviour — Bullwhip Effect

- ▶ **Customer spike:** creates retailer backorders
- ▶ **Retailer:** places larger printer orders
- ▶ **Manufacturer:** creates even larger part orders (BOM expansion)
- ▶ **Provider:** stock depleted and restocked repeatedly

Unexpected behaviours

- kit_piezas keeps returning to ~120 — that is the provider's restock target
- Part orders rejected when provider stock < requested quantity
- Chained CLI commands stop after one rejection — later purchases left unexecuted

Why it matters

These are features, not bugs.

Emergence is what makes this project interesting — simple local rules interact to produce behaviour that no single agent was programmed for.

The bullwhip effect was named in the 1960s. We reproduced it accidentally. That is exactly the point.

Live Demo + Reflection

Live Demo Plan

1. Reset the system
2. Run 2–3 days of holiday-rush
3. Show the narrative log updating
4. Point out one decision per agent
5. Show one chart from the completed 25-day run

Reflection

- Clear local prompts matter more than long prompts
- Pre-fetching state reduced wasted reasoning turns
- LLMs are useful for strategy; deterministic execution is better for repetitive low-level loops
- Hardest part: observability — without logs, emergent behaviour is impossible to explain

Backup: if agents stall, use logs/narrative_holiday-rush.md and the saved charts.

"Deterministic software handles execution. LLMs handle strategy. Well-defined interfaces keep everything auditable and under control."